Under the Hood

What actually happens when you type

```
user@terminator> ls | wc -l
```

Today we follow it all the way down to the kernel.

Continues the Terminator overview from last class

TODAY

What we cover today: 01 to 05

01
X11 GUI & tabs
Your own window

02
Commands
fork, execvp, wait

03
Multiline input
Quotes, \ and $(...)

04
Redirection
< and > via dup2

05
Pipes
cmd | cmd | cmd

06
multiWatch
Parallel live output

07
Hotkeys
User-configurable

08
Signals & jobs
Ctrl+C, Ctrl+Z, fg, bg

09
History
Shared, Ctrl+R search

10
Auto-complete
Tab finishes names

11
Bash scripts
Your own interpreter

ALL
Two modes
Threads or processes

UNDER EVERY BLOCK **System calls** the program's only door into the kernel

0 The GUI | 1 How bash works | 2 System calls | 3 Which calls & why | 4 Raw terminal

■ just learned ■ learned earlier today □ still to come today □ later in the course

PART 0

The GUI

How the Terminator window is built, continuing last class.

Covers 01 · X11 GUI & tabs

From last class

Terminator is an X client. It never touches the screen or keyboard itself.

It sends drawing requests; the X server sends back events.

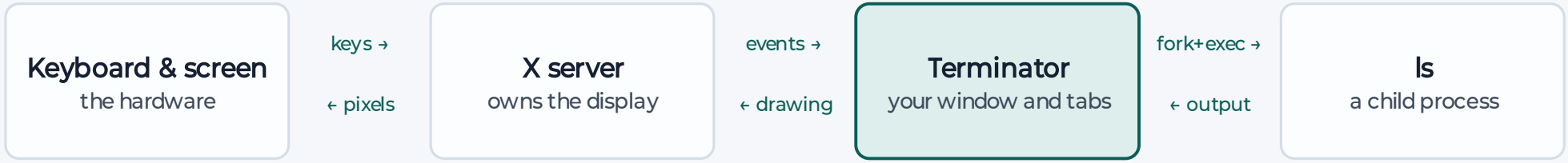
```
dpy = XOpenDisplay(NULL);       // connect
win = XCreateSimpleWindow(dpy, root,
        0, 0, 800, 600, 0, fg, bg);
XSelectInput(dpy, win, KeyPressMask |
        ExposureMask | StructureNotifyMask);
XMapWindow(dpy, win);           // show it
XNextEvent(dpy, &ev);          // wait for one
XDrawString(dpy, win, gc, x, y, s, n);
```

THE EVENTS TERMINATOR HANDLES

Event	When	Terminator does
KeyPress	a key is pressed	edit the input line; on Enter, run it
Expose	part of the window must be repainted	redraw from the tab's buffer
ConfigureNotify	the window is resized	recompute rows and columns, redraw
ButtonPress	a mouse click	switch tabs, select text

> Everything Terminator does starts as one of these events, or as output arriving on a pipe.

How Terminator is put together



YOU TYPE ls IN TAB 1 AND PRESS ENTER

1. Your keys go to the X server, which sends Terminator a KeyPress event for each one.
2. Terminator adds each letter to tab 1's input line and redraws that line.
3. On Enter, tab 1's shell parses "ls" and starts it with fork() and execvp().
4. ls writes its output into a pipe; Terminator reads the other end.
5. Terminator appends the text to tab 1's buffer and redraws the window.

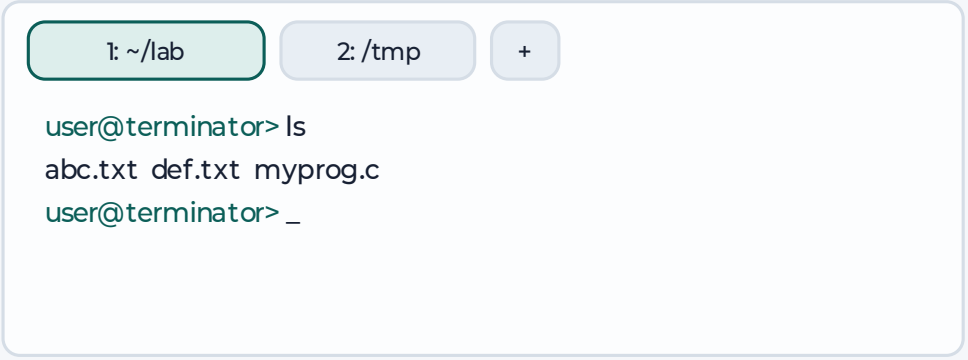
The screen is a drawing of your data

Change the data, then redraw. Never draw straight from a key press.

Each tab has its own buffer: output lines, the input line, the cursor, its directory.

Switching tabs only changes which buffer is drawn. Hidden tabs keep collecting output.

```
struct tab {
  char **lines;      // output so far
  char   input[4096];  // line being typed
  int    cursor;
  char   cwd[PATH_MAX]; // this tab's directory
  int    out_fd;     // pipe from the command
};
```



One loop runs the whole window

Wait, react, redraw. Terminator spends its whole life in this loop.

Two kinds of input. The X connection (keys, resize) and each tab's output pipe.

poll() waits on all of them at once and sleeps until one is ready: about 0% CPU when idle.

Never block inside the loop. A slow command must not freeze typing or other tabs.

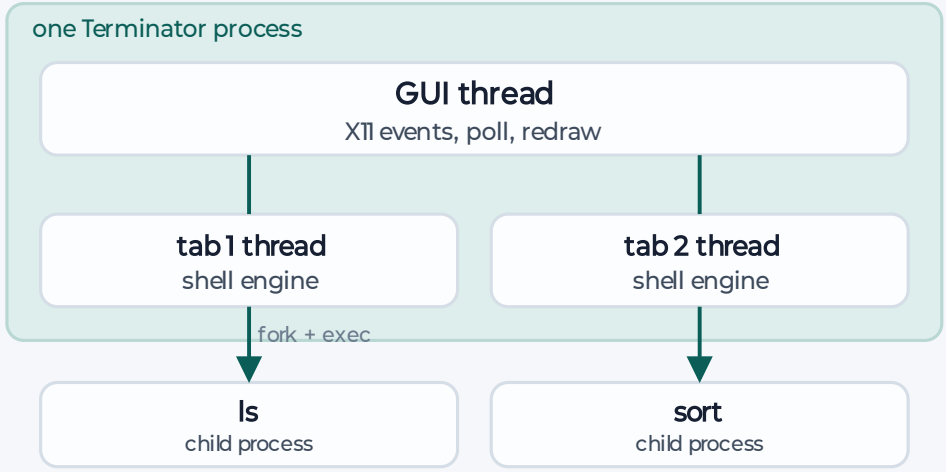
```
int xfd = ConnectionNumber(dpy);
for (;;) {
  while (XPending(dpy)) {    // keys, resize
    XNextEvent(dpy, &ev);
    handle_event(tab, &ev);
  }
  redraw_if_changed();

  poll(fds, nfds, -1);       // sleep here
  for each tab pipe that is ready:
    n = read(pipe, buf, sizeof buf);
    append_to_tab(tab, buf, n);
}
```

Tabs: threads or processes

THREAD MODE (default)

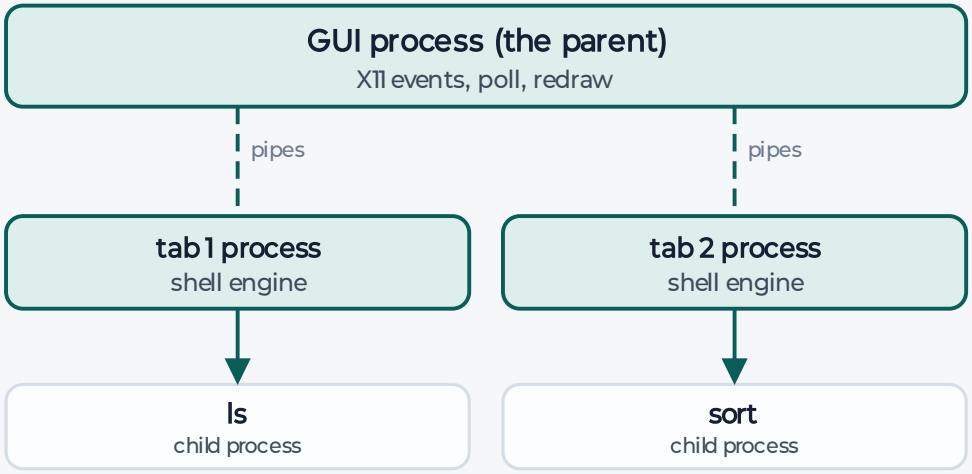
./terminator



Tabs share memory: history is a global, protected by a mutex.
Careful: chdir and dup2 affect every tab.

PROCESS MODE

./terminator useProcess=True



Tabs share nothing: history needs shared memory.
Bonus: a crashing tab can't take the window down.

CHECKPOINT · PART 0 DONE

You now know how the window works

01 ✓ JUST LEARNED
X11 GUI & tabs
poll loop + a buffer per tab

02
Commands
fork, execvp, wait

03
Multiline input
Quotes, \ and $(...)

04
Redirection
< and > via dup2

05
Pipes
cmd | cmd | cmd

06
multiWatch
Parallel live output

07
Hotkeys
User-configurable

08
Signals & jobs
Ctrl+C, Ctrl+Z, fg, bg

09
History
Shared, Ctrl+R search

10
Auto-complete
Tab finishes names

11
Bash scripts
Your own interpreter

ALL ✓ JUST LEARNED
Two modes
A thread or a process per tab

UNDER EVERY BLOCK **System calls** the program's only door into the kernel

✓ 0 The GUI | 1 How bash works | 2 System calls | 3 Which calls & why | 4 Raw terminal

Keys and command output both arrive in one poll() loop; each tab keeps its own buffer and the window is redrawn from it.

■ just learned ■ learned earlier today □ still to come today □ later in the course

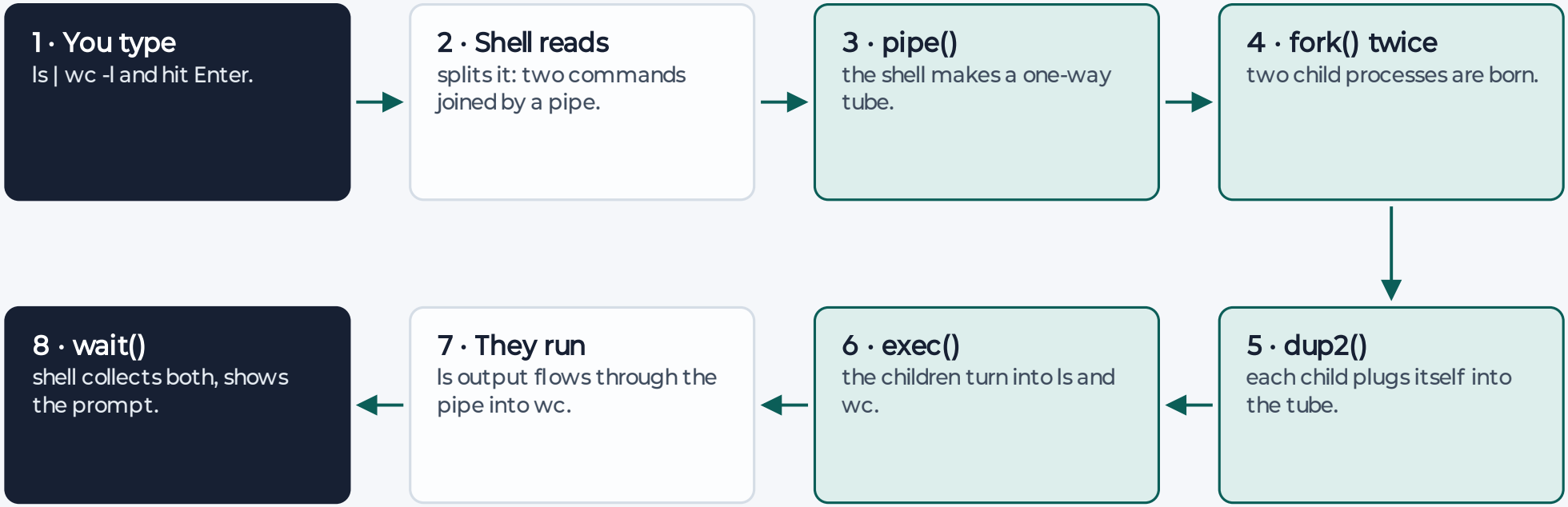
PART 1

How bash works

What a shell does with the line you type.

Covers 02 · Commands · 03 · Multiline input

You type ls | wc -l and press Enter. Then what?



> Steps 3 to 8 are system calls. The rest of today explains each one.

Every shell runs the same loop

Read, parse, run, repeat. bash, zsh and Terminator all work this way.

Built-ins run inside the shell. cd must: a child that calls chdir() changes only its own directory.

Everything else runs in a child process: fork, then exec, then wait.

$? holds the last exit status: 0 is success, 127 is command not found.

In Terminator the line comes from the GUI when the user presses Enter.

```
while (1) {
  print_prompt();
  line = read_line();
  cmd  = parse(line);
  if (is_builtin(cmd))
    run_builtin(cmd);    // cd, exit
  else {
    pid = fork();
    if (pid == 0)
      execvp(cmd[0], cmd);
    waitpid(pid, &status, 0);
  }
}
```

Reading a command line

```
sort < names.txt | uniq -c > counts.txt
```

Piece	Name	What the shell does
sort, uniq -c	words	a program and its arguments
< names.txt	input redirection	sort reads the file instead of the keyboard
|	pipe	sort's output becomes uniq's input
> counts.txt	output redirection	uniq writes to the file instead of the screen
"..." '...' \ $(...)	quotes, expansion	resolved before anything runs

When Enter is not the end of the command

Not finished yet? An open quote, a trailing \ or an unclosed $(means: show > and keep reading.

'single quotes' keep everything literal. "double quotes" keep spaces but still expand $VAR and $(...).

$(cmd) runs cmd and pastes its output in its place, innermost first.

```
user@terminator> echo "hello
> world"
hello
world
user@terminator> echo one \
> two
one two
user@terminator> echo "in $(basename $(pwd))"
in mydir
user@terminator> echo '$HOME' "$HOME"
$HOME /home/u
```

CHECKPOINT · PART 1 DONE

You now know what a shell must do

01 ✓
X11 GUI & tabs
Your own window

02 ✓ JUST LEARNED
Commands
Read → parse → run → wait

03 ✓ JUST LEARNED
Multiline input
Unfinished line? Keep reading

04
Redirection
< and > via dup2

05
Pipes
cmd | cmd | cmd

06
multiWatch
Parallel live output

07
Hotkeys
User-configurable

08
Signals & jobs
Ctrl+C, Ctrl+Z, fg, bg

09
History
Shared, Ctrl+R search

10
Auto-complete
Tab finishes names

11
Bash scripts
Your own interpreter

ALL ✓
Two modes
Threads or processes

UNDER EVERY BLOCK **System calls** the program's only door into the kernel

✓ 0 The GUI | ✓ 1 How bash works | 2 System calls | 3 Which calls & why | 4 Raw terminal

Terminator's shell copies bash: read a line, split it into commands, run them, wait, and keep reading if the line is not finished.

■ just learned ■ learned earlier today □ still to come today □ later in the course

PART 2

System calls

The only door into the kernel.

Used by every part of Terminator

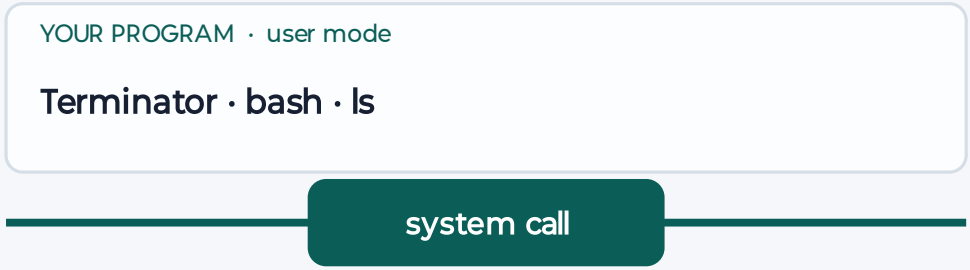
What is a system call?

Your program can compute, but it can't touch the disk, the screen or other processes.

A system call asks the kernel to do it: switch to kernel mode, do the work, return one number.

In a terminal: starting programs, reading output, opening files and connecting commands are all system calls.

Check the number: −1 means it failed, and perror() prints why.




```
printf("hi\n");       // library function
write(1, "hi\n", 3);   // the system call under it

if (fork() == -1)
   perror("fork");     // every call can fail
```

File descriptors are just numbered slots

Every process has numbered slots called file descriptors (fds).

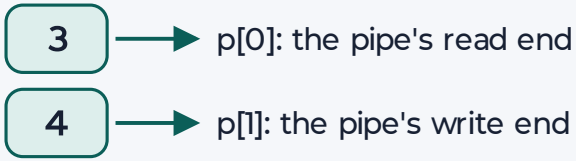
0 is stdin, 1 is stdout, 2 is stderr. Programs read 0 and write 1.

A new fd takes the lowest free number: the first pipe() gets 3 and 4.

dup2(old, new) makes slot new point where slot old points.

```
$ ls -l /proc/$$/fd
0 -> /dev/pts/0
1 -> /dev/pts/0
2 -> /dev/pts/0
```

AFTER pipe(p)



dup2(4, 1)	stdout now goes into the pipe.
dup2(3, 0)	stdin now comes from the pipe.

Watch bash do it with strace

```
$ strace -f -e trace=process,pipe2,dup2 \
    bash -c 'ls | wc -l'
pipe2([3, 4], 0)            = 0     // 1
clone(...)                  = 4312  // 2
clone(...)                  = 4313  // 2
[pid 4312] dup2(4, 1)       = 1     // 3
[pid 4313] dup2(3, 0)       = 0     // 4
[pid 4312] execve("/usr/bin/ls", ...)  // 5
[pid 4313] execve("/usr/bin/wc", ...)  // 5
wait4(-1, ...)              = 4312  // 6
wait4(-1, ...)              = 4313  // 6
```

1. bash makes a pipe: fd 3 read, fd 4 write.
2. two children are born. fork() is built on clone(), so strace shows clone.
3. child 1: stdout → pipe.
4. child 2: stdin ← pipe.
5. the children become ls and wc.
6. bash waits for both.

CHECKPOINT · PART 2 DONE

You now know the layer under every block

01 ✓
X11 GUI & tabs
Your own window

02 ✓
Commands
fork, execvp, wait

03 ✓
Multiline input
Quotes, \ and $(...)

04
Redirection
< and > via dup2

05
Pipes
cmd | cmd | cmd

06
multiWatch
Parallel live output

07
Hotkeys
User-configurable

08
Signals & jobs
Ctrl+C, Ctrl+Z, fg, bg

09
History
Shared, Ctrl+R search

10
Auto-complete
Tab finishes names

11
Bash scripts
Your own interpreter

ALL ✓
Two modes
Threads or processes

✓ JUST LEARNED UNDER EVERY BLOCK **System calls** the program's only door into the kernel

✓ 0 The GUI | ✓ 1 How bash works | ✓ 2 System calls | 3 Which calls & why | 4 Raw terminal

A system call is the only way a program asks the kernel for anything; file descriptors are the numbers those calls work on.

■ just learned ■ learned earlier today □ still to come today □ later in the course

PART 3

Which calls, and why

Exactly which system calls a terminal needs, with real code.

Covers 01 to 05

The system calls a terminal needs

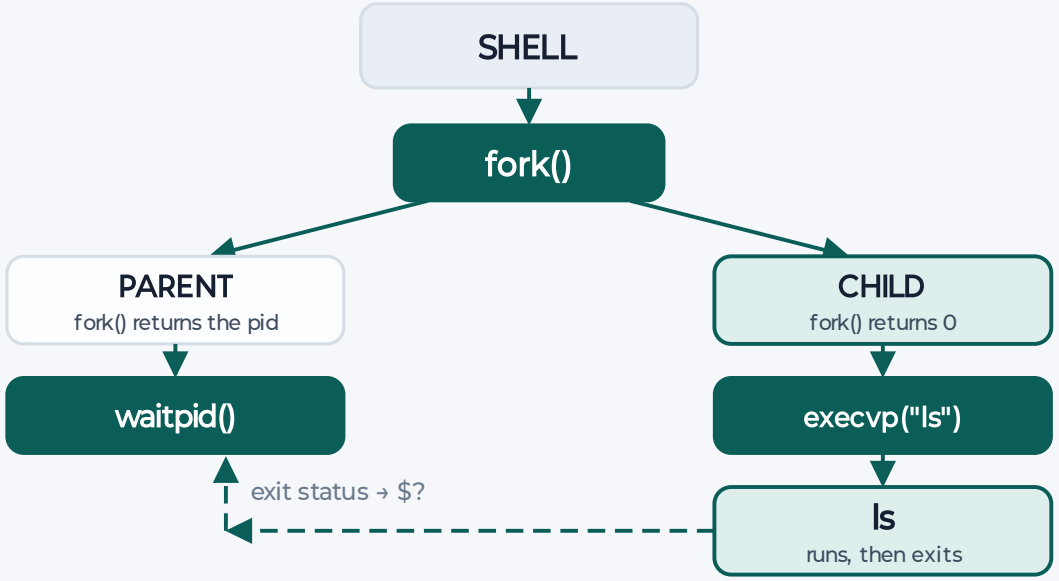
Call	What it does	Why Terminator needs it
fork()	copies the current process	run ls without replacing the shell
execvp()	replaces the program in this process	turn the copy into ls
waitpid()	waits for a child, gets its exit status	know when ls is done; set $?
pipe()	makes a one-way kernel buffer	connect ls to wc; get output into the tab
dup2()	makes one fd point where another points	redirect stdin / stdout to a file or a pipe
open() / close()	gives a file an fd / frees an fd	< and >; close unused pipe ends
read()	copies bytes out of an fd	read a command's output
poll()	sleeps until one of many fds is ready	wait for keys and output at the same time
chdir()	changes this process's directory	the cd built-in

Task by task: which calls, in which order

You are building…	Calls, in this order	Why this order
Running a command (ls -l)	fork → execvp (child) → waitpid (parent)	keep the shell alive, turn the copy into ls, then collect its exit status for $?
cd and pwd (built-ins)	chdir(path) → getcwd(buf, size)	must change the shell itself, so no fork; getcwd gives the new directory for the prompt
Output in the tab	pipe → fork → dup2 onto 1 and 2 → execvp; then poll → read → (read = 0) waitpid	the child writes into a pipe; the GUI sleeps in poll, reads the output, and reaps the child at EOF
Multiline: $(cmd)	pipe → fork → dup2 onto 1 → execvp; then read until 0 → waitpid	the shell itself needs cmd's output as text to paste into the line
Redirection (< in > out)	fork → open → dup2 → close → execvp	rewire fd 0 / 1 in the child, before the program starts
Pipes (a | b | c)	pipe ×(N−1) → fork ×N → dup2 → close unused ends → execvp → waitpid ×N	connect each stdout to the next stdin; the reader sees EOF only when every write end is closed

> Every call returns −1 on failure: check it, and print why with perror("name").

Running a command: fork, exec, wait



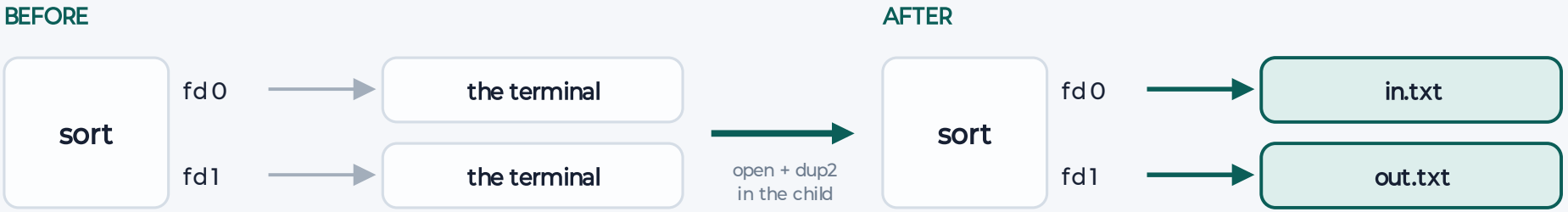

```
pid_t pid = fork();
if (pid == 0) {          // child
   execvp("ls", argv);
   perror("ls");         // exec failed
   _exit(127);
}
waitpid(pid, &status, 0);   // parent
last = WEXITSTATUS(status); // $?
```

fork creates the child; exec turns the child into the requested program; wait lets the shell collect it.

Why not just exec?

exec replaces whoever calls it. The shell itself would become ls and disappear. fork first, so one copy stays the shell.

Redirection: sort < in.txt > out.txt



Programs only know numbers.

sort reads fd 0 and writes fd 1. It doesn't know anything changed.

```
// in the child, after fork()
int in  = open("in.txt", O_RDONLY);          // missing? print error, don't run
int out = open("out.txt", O_WRONLY | O_CREAT | O_TRUNC, 0644);
dup2(in, 0);   dup2(out, 1);   close(in);   close(out);
execvp("sort", argv);
// >> out.txt: use O_APPEND instead of O_TRUNC
```

Pipes: ls | wc -l



N commands → N−1 pipes, N children

```
int p[2]; pipe(p);
if (fork() == 0) {              // ls
  dup2(p[1], 1); close(p[0]); close(p[1]);
  execvp("ls", a);
}
if (fork() == 0) {              // wc -l
  dup2(p[0], 0); close(p[0]); close(p[1]);
  execvp("wc", b);
}
close(p[0]); close(p[1]);       // the shell too!
waitpid(...); waitpid(...);
```

Close the ends you don't use

wc sees end-of-file only when every write end is closed. One forgotten close and wc waits forever.

Pipe + redirection

ls *.txt | sort > out.txt: the last child dup2s the file onto fd 1 instead of a pipe.

Getting a command's output into the tab

There is no terminal device in Terminator, so a command's output goes into a pipe.

The child points stdout and stderr at the pipe, then execs.

The GUI loop polls the other end. read() > 0: append to the tab. read() = 0: the command is done.

Then waitpid() to get $?, and show a new prompt.

$(cmd) works the same way, except the shell reads the pipe to the end and pastes the text into the line.

```
int out[2];  pipe(out);
if ((pid = fork()) == 0) {
   dup2(out[1], 1);   // stdout → pipe
   dup2(out[1], 2);   // stderr → pipe
   close(out[0]);  close(out[1]);
   execvp(argv[0], argv);
}
close(out[1]);

 // later, when poll() says out[0] is ready:
n = read(out[0], buf, sizeof buf);
if (n > 0)  append_to_tab(tab, buf, n);
if (n == 0) { waitpid(pid, &st, 0); prompt(); }
```

CHECKPOINT · PART 3 DONE

You now know the calls behind 01 to 05

01 ✓ JUST LEARNED
X11 GUI & tabs
poll → read each tab's pipe

02 ✓ JUST LEARNED
Commands
fork → execvp → waitpid

03 ✓ JUST LEARNED
Multiline input
$(cmd): pipe, fork, read

04 ✓ JUST LEARNED
Redirection
open → dup2 → close

05 ✓ JUST LEARNED
Pipes
pipe → dup2 → close

06
multiWatch
Parallel live output

07
Hotkeys
User-configurable

08
Signals & jobs
Ctrl+C, Ctrl+Z, fg, bg

09
History
Shared, Ctrl+R search

10
Auto-complete
Tab finishes names

11
Bash scripts
Your own interpreter

ALL ✓
Two modes
Threads or processes

✓ UNDER EVERY BLOCK **System calls** the program's only door into the kernel

✓ 0 The GUI | ✓ 1 How bash works | ✓ 2 System calls | ✓ 3 Which calls & why | 4 Raw terminal

Each of today's blocks is a short, fixed sequence of calls, made in a fixed order. This is the checklist to code against.

■ just learned ■ learned earlier today □ still to come today □ later in the course

PART 4

The raw terminal

What the kernel does with your keys, and what Terminator must do itself.

Covers 01 · X11 GUI & tabs · 03 · Multiline input

What a terminal really is

Teletype (TTY) → VT100 video terminal (1978) → a terminal emulator program

GNOME TERMINAL + BASH

GNOME Terminal
draws pixels, reads keys

⇄

Kernel: pty + line discipline
collects the line, echoes, turns Ctrl+C into SIGINT

⇄

bash and its commands
fds 0, 1, 2 → /dev/pts/3

TERMINATOR

X server
key events in, pixels out

⇄

Terminator: your code
draws, edits the line, runs commands

⇄

your commands
fds 0, 1, 2 → pipes

> Type tty in any terminal to see your device. In Terminator there is none: you do its job.

Cooked mode vs raw mode

	Cooked (the default)	Raw
Input arrives	a whole line, after Enter	each key, immediately
Backspace, Ctrl+U	handled by the kernel	your program handles them
Echo of typed keys	the kernel echoes	your program draws them
Ctrl+C / Ctrl+Z	kernel sends SIGINT / SIGTSTP	just bytes 0x03 / 0x1A
Ctrl+D	end of file	just byte 0x04
Who uses it	cat, simple read() loops	vim, less, bash's line editor

```
$ stty -a          # show the settings
$ stty raw -echo   # go raw
$ stty sane        # recover
```

Terminator is always in the raw column: X11 gives you one key at a time.

Entering raw mode with termios

```
struct termios orig, raw;
tcgetattr(STDIN_FILENO, &orig);
raw = orig;
raw.c_lflag &= ~(ICANON | ECHO | ISIG | IEXTEN);
raw.c_iflag &= ~(IXON | ICRNL);
raw.c_oflag &= ~(OPOST);
raw.c_cc[VMIN] = 1;  raw.c_cc[VTIME] = 0;
tcsetattr(STDIN_FILENO, TCSAFLUSH, &raw);

char c;
while (read(STDIN_FILENO, &c, 1) == 1 && c != 'q')
  printf("%d\r\n", c);

tcsetattr(STDIN_FILENO, TCSAFLUSH, &orig);  // restore
```

ICANON off: no line buffering

ECHO off: keys aren't echoed

ISIG off: Ctrl+C, Ctrl+Z are just bytes

IXON off: Ctrl+S, Ctrl+Q free

ICRNL off: Enter stays \r

OPOST off: print \r\n yourself

VMIN 1: read returns after 1 byte

Always restore the old settings, even on a crash path (atexit).

Without a terminal device, you do its job

In GNOME Terminal, the kernel...	In Terminator, you...
echoes the keys you type	draw them from the tab's input line
handles Backspace and Ctrl+U	edit the input line and cursor yourself
delivers the line on Enter	hand it to your parser, or show > and keep reading
connects the program to the screen	give it a pipe and read the other end
sends SIGINT on Ctrl+C (later)	kill(-pgid, SIGINT) the running command

> Commands see a pipe, not a terminal, so ls prints one name per line. That is correct.

CHECKPOINT · PART 4 DONE

You now know who handles the keys

01 ✓ JUST LEARNED
X11 GUI & tabs
No tty: you echo and edit keys

02 ✓
Commands
fork, execvp, wait

03 ✓ JUST LEARNED
Multiline input
Enter: run it, or show > and wait

04 ✓
Redirection
< and > via dup2

05 ✓
Pipes
cmd | cmd | cmd

06
multiWatch
Parallel live output

07
Hotkeys
User-configurable

08
Signals & jobs
Ctrl+C, Ctrl+Z, fg, bg

09
History
Shared, Ctrl+R search

10
Auto-complete
Tab finishes names

11
Bash scripts
Your own interpreter

ALL ✓
Two modes
Threads or processes

✓ UNDER EVERY BLOCK **System calls** the program's only door into the kernel

✓ 0 The GUI | ✓ 1 How bash works | ✓ 2 System calls | ✓ 3 Which calls & why | ✓ 4 Raw terminal

There is no terminal device, so echo, Backspace, Enter and Ctrl+C are all your code. That completes 01 to 05.

■ just learned ■ learned earlier today □ still to come today □ later in the course

What should work after today

Part	It works when…	System calls
01 Your own window	keys appear as you type; output shows up without freezing	poll, read, pipe
02 Commands	ls -l runs; cd changes the tab; unknown commands print an error	fork, execvp, waitpid, chdir
03 Multiline input	an open quote, a trailing \ and $(pwd) all work	pipe, fork, read, waitpid
04 Redirection	sort < in.txt > out.txt works; a missing file is reported	open, dup2, close
05 Pipes	ls *.txt | sort | head -5 works and finishes	pipe, dup2, close

> man 2 fork · execve · waitpid · pipe · dup2 · open · poll · man 3 termios · Kerrisk, The Linux Programming Interface

Terminator deadlines

TODAY · Wed 7 Oct

CHECKPOINT 1

17 Oct

Saturday

10 days from today

CHECKPOINT 2

30 Oct

Friday

23 days from today

FINAL SUBMISSION

3 Nov

Tuesday, end of day

27 days from today